

虛擬電廠即時排程系統

Virtual Power Plant Real-Time Scheduling (VPP-RTS)

即時系統 (Real-Time Systems) 作業

完整實作報告 + 報告講稿

Level 1：日前排程 · 接收測試 · 效能評估

Level 2：放寬假設建模 · 進階動態排程

國立成功大學 即時系統課程

本文件用途

本報告分為兩大部分：

- **第一部分「完整實作說明」**：讓未參與開發的組員，讀完即可掌握整份作業的問題定義、數學模型、四個階段的演算法、Level 2 的放寬假設與動態排程，以及實際執行結果。
- **第二部分「報告講稿」**：依投影片順序撰寫，可直接照著對老師口頭報告（含開場、各段重點、預期問答）。

目錄

本文件用途	1
第一部分 完整實作說明	2
1 作業背景與目標	2
1.1 三類發電設備（處理器）	2
1.2 三類用電需求（任務）	2
1.3 整體目標函數（三個彼此衝突的目標）	2
2 系統輸入與資產設定	3
2.1 傳統機組（2 部）	3
2.2 再生能源與儲能	3
2.3 市場價格	3
3 系統架構與管線（Pipeline）	3
4 Phase 1：週期任務集生成	4
4.1 本次產生的週期任務集（8 個任務，frame size = 4）	4
5 Phase 2：日前 MILP 排程	5
5.1 決策變數	5
5.2 核心限制式	6
5.3 23 條限制式總覽（依實作分組）	6
5.4 目標函數與求解	7
6 Phase 3：接收測試（Sporadic / Aperiodic）	7
6.1 本次靜態接收測試結果	7

目錄	2
7 Phase 4：效能評估指標	7
8 Level 2-A：放寬假設建模（R1–R10）	8
8.1 放寬參數表（本次採用值）	8
8.2 放寬限制式 R1–R10	9
9 Level 2-B：進階動態排程（滾動視窗再最佳化）	9
9.1 核心機制	10
9.2 目標衝突（為何無法同時最佳化三目標）	10
10 實際執行結果與靜態 vs 動態比較	10
10.1 動態排程的 9 輪再最佳化（共 9 次 MILP 解，約 12 秒，單執行緒、可重現）	11
10.2 靜態（L1）vs 動態（L2）效能比較	11
11 程式碼結構與執行方式	12
12 對應評分表的自我檢核	13
第二部分 報告講稿	14

第一部分 完整實作說明

1 作業背景與目標

「虛擬電廠 (Virtual Power Plant, VPP)」把分散的發電與儲能資產整合成一個可被統一調度的電力系統。本作業要為一個 VPP 設計一套即時排程系統：一方面安排各種「發電設備」供電，另一方面滿足一連串有時間限制的「用電需求 (job)」。

我們把發電設備視為即時系統中的**處理器(processor)**、把用電需求視為**任務(task/job)**，於是電力調度問題就被轉化成一個即時排程問題。時間軸為 $H = 72$ 小時，每格 $\Delta t = 1$ 小時。

1.1 三類發電設備（處理器）

- **傳統機組** I_g ：可控輸出的火力機組，有開關機、最小開 / 關機時間、爬升率與成本。
- **再生能源** I_r ：太陽能，輸出受日前預測曲線上限約束。
- **儲能裝置** I_b ：電池，可充可放但不可同時，受 SOC 上下限約束。

1.2 三類用電需求（任務）

- **週期任務** J_p ：固定週期釋放的硬截止任務（例：工廠定時製程）。
- **零星任務** J_s ：硬截止、臨時出現，須通過「接收測試」才接受（例：緊急冷卻）。
- **非週期任務** J_a ：軟截止、可等待，逾時僅記為 miss 與 tardiness（例：非緊急充電）。

1.3 整體目標函數（三個彼此衝突的目標）

系統以最小化形式同時追求「少漏非週期任務、低機組成本、高售電收益」：

$$\min F = \alpha f_1 + f_2 + f_3 \quad (1)$$

$$f_1 = \sum_{j \in J_a} \text{Miss}_j, \quad \alpha = 10000 \text{ (\$/miss)} \quad (2)$$

$$f_2 = \sum_{i \in I_g} \sum_{t \in T} \left(\text{coston}_i \cdot \min(1, P_{i,t}) + \text{costup}_i \cdot P_{i,t} \right) \quad (3)$$

$$f_3 = - \sum_{t \in T} \lambda_t \text{Sell}_t \quad (4)$$

其中 f_1 為非週期逾時數（以罰係數 α 轉成金額）， f_2 為機組成本， f_3 為售電收益取負號（最小化 f_3 即最大化收益）。

2 系統輸入與資產設定

輸入檔 `input/processor_settings.json` 與 `input/price_72hr.json` 定義固定的資產與價格。本次使用的資產如下。

2.1 傳統機組（2 部）

機組	出力下/上限 (MW)	爬升 RU/RD	最小開/關 (h)	固定 (\$/h)	變動 (\$/MWh)
thermal_1	15 / 80	15 / 15	3 / 2	1200	42
thermal_2	10 / 45	20 / 20	2 / 2	600	70

thermal_1 變動成本低（基載）；*thermal_2* 啟動便宜但每 *MWh* 較貴（尖峰備用）。兩部初始皆關機。

2.2 再生能源與儲能

設備	型別	關鍵參數
pv_1	太陽能	額定 60 MW，依日前預測比例出力
pv_2	太陽能	額定 80 MW，預測曲線每日三段日照尖峰（約 8–13、31–43、55–66 時）
battery_1	儲能	SOC 20–100 MWh，充/放 20 MW，初始 45 MWh
battery_2	儲能	SOC 10–60 MWh，充/放 15 MW，初始 25 MWh

每個電池對應一個充電需求（*battery_1_chg* / *battery_2_chg*），充電只能由機組或再生能源供應，不能由其他電池放電供應。

2.3 市場價格

`price_72hr.json` 提供每小時售電價 λ_t 。本資料尖峰落在第 70 時（113）、第 22 時（108）、第 56 時（105），谷底為第 12 時（0）。排程因此傾向把多餘電力留到高價時段售出。

3 系統架構與管線（Pipeline）

整個系統由 `src/main.py` 串接四個階段；Level 2 另以 `run_level2()` 串成「生成 → 靜態 L1 → 動態 L2 → 比較」。

Phase 1	任務生成	generator/	-> output/task_set.json
Phase 2	日前排程	rt_scheduler/	-> output/schedule_result.json (PuLP MILP)
Phase 3	接收測試	acceptance_tester	-> 在排程上標註接受/拒絕 (整合於 Phase 2 之後)
Phase 4	效能評估	evaluator/	-> output/evaluation_results.json
Level 2	動態排程	advanced_scheduler	-> output/*_dynamic.json

設計上採物件導向與單一職責：生成器把計算交給 FrameSizeCalculator、驗證交給 TaskSetValidator、排程器把建模交給 VppMipFormulator、結果解析交給 SchedulerResultExtractor。資料模型以 Pydantic 定義，I/O 集中於 JsonIO。

4 Phase 1：週期任務集生成

TaskSetGenerator 以「生成 → 計算 frame size → 驗證」迴圈，隨機產生一組同時滿足所有作業限制的週期任務；不合格即重抽，直到通過 TaskSetValidator。為保證較難的約束，生成策略刻意：

- 先生成 $d = e$ 的任務（保證評分 1-6： $\geq 20\%$ 任務 $d_j = e_j$ ）。
- 再生成 2 個 $e = 2$ 的非搶占任務（保證評分 1-7： ≥ 2 個 $e \neq 1$ 的非搶占任務）。
- 其餘任務隨機補足，並維持週期種類 ≥ 3 。

Frame size f 取最小可行值，需同時滿足：

$$f \geq \max_j e_j, \quad H \bmod f = 0, \quad 2f - \gcd(f, p_j) \leq d_j \quad \forall j. \quad (5)$$

4.1 本次產生的週期任務集（8 個任務，frame size = 4）

ID	r 釋放	p 週期	e 執行	d 截止	w (MWh)	搶占	job 數
p1	20	20	4	4	11	否	3
p2	1	10	2	8	17	否	7
p3	17	20	4	4	8	否	3
p4	8	8	1	5	13	否	8
p5	8	24	2	19	17	否	2
p6	11	13	2	11	17	否	4
p7	10	16	2	13	16	否	4
p8	9	9	2	7	13	否	7

合計展開為 38 個週期 job (> 30 ✓)；密度 $D_W = \sum_j e_j/p_j = 1.31 \geq 0.7$ ✓；週期 7 種 (≥ 3 ✓)； $d = e$ 者為 p1、p3 ($2/8 = 25\% \geq 20\%$ ✓)；非搶占且 $e \neq 1$ 者有 p2、p5、p6、p7、p8 (≥ 2 ✓)； $f = 4$ 通過上式 (p8 最緊： $2f - \gcd(4, 9) = 7 \leq d_{p8} = 7$) ✓。本次實例所有任務皆為非搶占。

展開規則：第 k 個 job 的 $\text{release} = r + kp$ ， $\text{deadline} = \text{release} + d - 1$ ，僅保留 $\text{deadline} \leq 72$ 者。此外 `task_set.json` 亦含 Demo 提供的 6 個零星任務 (s1–s6) 與 10 個非週期任務 (a1–a10)，供 Phase 3 使用。

5 Phase 2：日前 MILP 排程

`VppMilpFormulator` 把 72 小時排程建成一個混合整數線性規劃 (MILP)，以 PuLP + CBC 一次解出整個 horizon。

5.1 決策變數

變數	意義
$P_{i,t}$	設備 i 在 t 的總出力 (MWh)
$k_{j,i,t}$	設備 i 在 t 供給需求 j 的能量 (充電需求則為對電池的充電量)
$u_{i,t}$, $\text{start}_{i,t}$, $\text{stop}_{i,t}$	機組開關機 / 啟動 / 停機二元變數 (線性化 $\min(1, P)$)
$\text{charge_b}_{i,t}$, $\text{discharge_b}_{i,t}$	電池充 / 放電二元旗標 (互斥)
$\text{SOC}_{i,t}$	電池在 t 末的儲能狀態
Sell_t	t 時售電量
$x_{j,t}$	需求 j 在 t 是否執行之二元變數 (線性化 $\min(1, \sum k)$)

規格中的 $\min(1, \cdot)$ 為非線性，實作以二元變數 x (需求是否啟動) 與 u (機組是否開機) 線性化——這是把規格轉成可解 MILP 的關鍵。

5.2 核心限制式

任務完成性與供需平衡是模型骨幹，以數學式列出幾條代表：

$$(C1) \text{ 啟動時收滿需求： } \sum_{i \in I} k_{j,i,t} = w_j x_{j,t}, \quad \forall j, \forall t \quad (6)$$

$$(C3) \text{ 截止前累積執行 } e \text{ 格： } \sum_{t=r_j}^{r_j+d_j-1} x_{j,t} = e_j, \quad \forall j \in J_p \cup J_s \quad (7)$$

$$(C7) \text{ 爬升率： } P_{i,t} - P_{i,t-1} \leq RU_i \Delta t, \quad P_{i,t-1} - P_{i,t} \leq RD_i \Delta t, \quad \forall i \in I_g \quad (8)$$

$$(C13) \text{ 再生能源上限： } P_{i,t} \leq \text{renewmax}_i \cdot \text{forecast}_{i,t} \cdot \Delta t, \quad \forall i \in I_r \quad (9)$$

$$(C16) \text{ SOC 平衡： } SOC_{i,t} = SOC_{i,t-1} + \sum_{\text{charge}} k - P_{i,t}, \quad \forall i \in I_b \quad (10)$$

$$(C23) \text{ 供需平衡： } \sum_{i \in I} P_{i,t} = \sum_j \sum_i k_{j,i,t} + \text{Sell}_t, \quad \forall t \quad (11)$$

5.3 23 條限制式總覽（依實作分組）

群組	編號	內容
任務	C1–C3, C5	啟動收滿 w ；釋放前不執行；截止前累積 e 格；非搶占須連續 ($0 \rightarrow 1$ 上升沿 ≤ 1)
非週期	C4	Miss_j 定義：未逾時即完成 e 格，否則記 $\text{Miss} = 1$
機組	C6–C12	出力上下限、爬升率、下限 $\leq$ 單格爬升、最小開/關機、初始開/關機延續
再生能源	C13	$P \leq \text{capacity} \cdot \text{forecast} \cdot \Delta t$
儲能	C14–C19, C21	充/放上限、SOC 平衡、SOC 上下限、放電 $\leq$ 可用儲能、不可同時充放、充電不可由非發電設備供應
售電	C22	$\text{Sell}_t \geq 0$
平衡	C20, C23	各設備供給 $\leq$ 其出力；每小時總發電 = 需求 + 充電 + 售電

$C16$ 與 $C18$ 在 $Level\ 2$ 由「真實儲能」放寬版取代（見第 8 節），預設參數下與原式等價。

5.4 目標函數與求解

目標即第 1 節的 F 。Level 1 的 MILP 只將週期 job 與充電 job 納入模型；零星 / 非週期任務由 Phase 3 以保留量處理，故靜態 MILP 求解時 $f_1 = 0$ ，非週期逾時罰則於 Phase 4 評估時才計入 `objective_value`。求得最佳解後，`SchedulerResultExtractor` 解析各變數、四捨五入、捨去近零值，輸出 `schedule_result.json`。

6 Phase 3：接收測試 (Sporadic / Aperiodic)

`AcceptanceTester` 在 MILP 解出後執行（已整合進 `RTScheduler.run()`）。它利用 MILP 留下的保留量——原本要賣到市場的剩餘電力 $Sell_t$ ，可拆解為各設備的 `spare`：

$$spare_{i,t} = P_{i,t} - \sum_j k_{j,i,t}, \quad \sum_i spare_{i,t} = Sell_t \text{ (由 C23)}. \quad (12)$$

接受 job 時，把其每格需求 w 從 `spare` 轉入 $k_{job,i,t}$ ，並同額減少 $Sell_t$ ——總發電不變，C23、C20 仍成立。決策規則：

- **零星（硬截止）**：唯有 $[r, r + d - 1]$ 內能排入 e 格、每格保留量 $\geq w$ 時才接受，否則拒絕。
- **非週期（軟截止）**：盡量排到 H 前完成；若完成時刻晚於軟截止則另記為 `miss`。
- 搶占 = 1 可用不連續時槽；搶占 = 0 須單一連續視窗。

每個決策（接受/拒絕、排入時槽、完成時刻、理由）寫入 `acceptance_test_log.json`，支援評分 4-1（方法）與 4-2（合理性）。

6.1 本次靜態接收測試結果

零星 `s1–s6` 全部接受（`value rate = 1.0`）：例如 `s1` 在 $[2, 9]$ 排入第 2、4 時；`s3` 在 $[28, 33]$ 排入第 28、29 時。非週期 `a1–a10` 全部排入，但 `a2`（完成於 21 > 軟截止 17）與 `a5`（完成於 46 > 軟截止 42）逾時，故靜態軟逾時率 = $2/10 = 0.2$ 。

7 Phase 4：效能評估指標

`Evaluator` 讀入排程結果與輸入檔，計算所有規格指標，輸出 `evaluation_results.json`：

指標	定義
hard_deadline_miss_rate	(逾時的週期 + 零星 job) / 總硬截止 job 數
soft_deadline_miss_rate	逾時的非週期 job / 總非週期 job 數
average / max_tardiness	$T_j = \max(0, C_j - d_j)$ 的平均與最大
average / max_response_time	$R_j = C_j - r_j$ 的平均與最大
completion_time_jitter	同一週期任務各 instance 完成時刻的峰對峰值，再取平均
sporadic_value_rate	硬截止前完成的零星執行格數 / 零星總執行格數
generator_cost / market_revenue	f_2 機組成本 / $\sum_t \lambda_t \text{Sell}_t$ 售電收益
objective_value	$F = \alpha \cdot (\text{軟逾時數}) + \text{機組成本} - \text{售電收益}$

8 Level 2-A：放寬假設建模（R1–R10）

我們放寬三個 Level 1 假設——假設 11（再生能源預測必準）、假設 12（理想儲能）、假設 5（無工作優先序）——並嚴格遵守規格：可修改符號、可加參數，但不得新增決策變數。下列每條放寬限制都只用 Level 1 既有變數 $P, k, \text{SOC}, \text{Sell}, x$ 表達。所有參數集中於 runtime_config.json；全取預設（ $\eta = 1, \sigma = 0, \beta = 0, \dots$ ）時模型與 Level 1 完全相同。

8.1 放寬參數表（本次採用值）

參數	符號	值	假設	意義
charge_efficiency	η_c	0.95	12	充電能量僅 η_c 進入電芯
discharge_efficiency	η_d	0.95	12	放出 P 需從電芯抽 P/η_d
self_discharge_rate	σ	0.002	12	每格自放電比例
cycle_limit	N	3.0	12	全期最大等效循環數
soc_power_floor	φ	0.5	12	SOC 極端時最低功率比例
aging_cost	c_{age}	2.0	12	每放電 1 MWh 老化成本 (\$)
renewable_uncertainty_margin	β	0.15	11	預測折減保留邊際
precedence	—	自動	5	有序工作對 (a, b)

8.2 放寬限制式 R1–R10

假設 11 ——再生能源不確定性

$$(R1 / C13') \quad P_{i,t} \leq \text{renewmax}_i \cdot \text{forecast}_{i,t} \cdot (1 - \beta) \cdot \Delta t, \quad \forall i \in I_r. \quad (13)$$

日前計畫不靠滿預測，保留邊際以防高估； $\beta = 0$ 還原 C13。

假設 12 ——真實儲能（就地改寫 C16、C18，預設參數即還原）

$$(R2\text{--}R4 / C16') \quad \text{SOC}_{i,t} = (1 - \sigma) \text{SOC}_{i,t-1} + \eta_c \text{charge_in}_{i,t} - \frac{1}{\eta_d} P_{i,t} \quad (14)$$

$$(R5 / C18') \quad \frac{1}{\eta_d} P_{i,t} \leq (1 - \sigma) \text{SOC}_{i,t-1} - \text{soc_min}_i \quad (15)$$

$$(R6 \text{ 循環/吞吐}) \quad \sum_t P_{i,t} \leq N (\text{soc_max}_i - \text{soc_min}_i) \quad (16)$$

$$(R7 \text{ SOC 相依放電}) \quad P_{i,t} \leq \text{dis}_i \Delta t \left[\varphi + (1 - \varphi) \frac{\text{SOC}_{i,t-1} - \text{soc_min}_i}{\text{soc_max}_i - \text{soc_min}_i} \right] \quad (17)$$

$$(R8 \text{ SOC 相依充電}) \quad \text{charge_in}_{i,t} \leq \text{chg}_i \Delta t \left[\varphi + (1 - \varphi) \frac{\text{soc_max}_i - \text{SOC}_{i,t-1}}{\text{soc_max}_i - \text{soc_min}_i} \right] \quad (18)$$

$$(R9 \text{ 老化成本，進目標}) \quad F += \sum_{i \in I_b} \sum_t c_{\text{age}} P_{i,t}. \quad (19)$$

R7/R8 使功率在接近滿電時達額定、接近不利端時降到 φ ·額定，線性內插。

假設 5 ——工作優先序

$$(R10) \quad e_a x_{b,t} \leq \sum_{s < t} x_{a,s}, \quad \forall t \in b \text{ 的視窗}. \quad (20)$$

即 a 全部做完前 b 不得啟動；重用 x ，無新變數。未設定時程式自動挑一組可行的非平凡對（本次為 $p2_0 \rightarrow p4_0$ ）。

保留策略（支援評分 6/7 與動態法）

$$(R\text{-reserve}) \quad \text{Sell}_t \geq R_t. \quad (21)$$

以對 Sell 設下限強制計畫保留可轉移剩餘，供接收測試使用；重用 Sell ，無新變數。

合計 10 條放寬限制（R1–R10）外加保留下限，達到評分 3 的上限 10 分，且皆未新增決策變數。

9 Level 2-B：進階動態排程（滾動視窗再最佳化）

方法：Receding-horizon（滾動視窗）再最佳化。 Level 1 在 $t = 0$ 一次解完整個 $[1, 72]$ ；AdvancedScheduler 則沿時間前進，在觸發點對「剩餘 horizon」重新最佳化，因應靜態計畫看不到的動態資訊。

9.1 核心機制

- **狀態承接 (pin_prefix)**：每次觸發時，已執行的 $[1, t_0)$ 被釘住為已承諾值（連續變數 $P, k, \text{SOC}, \text{Sell}$ 加上由其推導的前綴二元變數 $u, \text{charge_b}, \text{discharge_b}, x$ ），只重解 $[t_0, 72]$ 。釘住前綴讓 solver 的 presolve 將其消去，使每次再解都便宜，且機組開關機時長、爬升位置、SOC 都正確跨界承接。
- **觸發時機**：(a) 週期節奏 $\text{reopt_interval} = 24$ 格；(b) 每個零星任務釋放（硬任務需即時審核）；(c) 最多 3 個「實際 PV 與預測偏差最大」的時點。非週期任務於下一個節奏邊界揭露。
- **再生能源實現**：以種子化隨機模型（noise std = 0.2, seed = 42）在預測周圍生成「實際」PV。已承諾區塊用實際值、未見尾段用折減預測 (R1) ——對已承諾精確、對未來保守。
- **線上接收 = 保留可行性**：任務於釋放時揭露，能否接受取決於再最佳化「能否在其執行視窗內保留足夠可轉移剩餘 ($\text{Sell} \geq R_t$)」。硬任務不可行即拒絕，軟任務盡力保留。被接受者隨區塊凍結被路由進排程，故接受的硬任務必在截止前完成。

每次再解使用 $MIP\ gap$ ($gap_rel = 0.05$) 與時間上限 ($time_limit = 20s$)，確保線上再最佳化可解。

9.2 目標衝突（為何無法同時最佳化三目標）

- 提高接收（降 miss） $\Rightarrow$ 須保留更多剩餘 $\Rightarrow$ 更多機組承諾 $\Rightarrow$ 機組成本 f_2 上升，或售電收益下降（剩餘被留著而非在尖峰賣出）。
- 再生能源不確定（R1 折減 + 實際短缺） $\Rightarrow$ 能量由 PV 移到火力 $\Rightarrow f_2$ 上升。
- 儲能真實化（效率、自放電、循環、老化） $\Rightarrow$ 電池成為更貴、更有損的緩衝 $\Rightarrow$ 套利收益變少。

因此動態法是「以較高機組成本換取更少逾時與更佳即時回應」，下一節以實際數據驗證。

10 實際執行結果與靜態 vs 動態比較

10.1 動態排程的 9 輪再最佳化（共 9 次 MILP 解，約 12 秒，單執行緒、可重現）

觸發 t_0	承諾區間	本輪揭露	結果
1	[1, 1]	(無)	建立初始計畫
2	[2, 13]	s1, a1, a2	全部接受並路由
14	[14, 24]	s2, a3	全部接受
25	[25, 27]	a4	接受
28	[28, 39]	s3, a5	全部接受
40	[40, 48]	s4, a6	全部接受
49	[49, 52]	a7	接受
53	[53, 65]	s5, a8, a9	全部接受
66	[66, 72]	s6, a10	全部接受

9 輪皆無拒絕、無再排程失敗；38 個週期 job、6 個零星 job 全部於硬截止前完成，10 個非週期 job 全部於軟截止前完成。逐格驗證：C23 平衡 0 違反、SOC 維持於 $[\text{soc_min}, \text{soc_max}]$ 0 違反、放寬儲能/機組/再生能源限制每輪皆成立。

10.2 靜態 (L1) vs 動態 (L2) 效能比較

指標	靜態 L1	動態 L2	差異 Δ
objective_value	-69 410.4	-83 238.8	-13 828.4 (更佳)
generator_cost	296 403.6	311 428.9	+15 025.3
market_revenue	385 814.0	394 667.8	+8 853.8
hard_deadline_miss_rate	0.0	0.0	—
soft_deadline_miss_rate	0.2	0.0	-0.2
average_tardiness	0.15	0.0	-0.15
max_tardiness	4	0	-4
average_response_time	3.63	2.65	-0.98
sporadic_value_rate	1.0	1.0	—

解讀：

- 靜態日前計畫雖一次看到所有任務，但只承諾一份排程，漏掉 2 個非週期任務(a2、a5，軟逾時率 0.2)。動態法隨任務到達再最佳化並為其保留容量，10 個全部準時(軟逾時率 0.0)，平均回應時間由 3.63 降到 2.65。
- 消除 2 個軟逾時 = 省下 $2\alpha = 20\,000$ 罰則，主導目標改善 ($-69\,410 \rightarrow -83\,239$)。
- 代價清晰可見：保留容量並補償實際再生能源短缺使機組成本上升 (+15 025)；較大的承諾發電同時讓售電收益增加 (+8 854)。扣掉逾時罰則後，動態法在總目標上明顯更佳，印證上節的權衡分析。

11 程式碼結構與執行方式

```
src/
main.py           管線進入點 (run_level2 = L1 + L2 + 比較)
config.py         路徑與常數
validator.py      自我檢核 (純標準庫)
advanced_scheduler.py  Level 2 滾動視窗動態排程
generator/        Phase 1 任務生成
rt_scheduler/     Phase 2+3: formulator / acceptance_tester /
                  relaxation / expander / extractor
evaluator/        Phase 4 評估
model/            Pydantic 資料模型
input/  processor_settings.json, price_72hr.json
output/ task_set.json, schedule_result*.json, evaluation_results*.json,
        acceptance_test_log*.json, dynamic_run_log.json
runtime_config.json  Level 2 放寬 + 動態節奏參數
```

執行指令：

```
# 環境
pip install pulp pydantic          # 或 poetry install

# Level 1 完整管線 (生成 -> 排程 -> 評估)
python -m src.main

# Level 2 比較管線 (生成 -> 靜態 -> 動態 -> 印出比較)
python -c "from src.main import run_level2; run_level2()"

# 只跑動態排程 / 自我檢核
python -m src.advanced_scheduler
```

```
python3 -m src.validator          # Level 1
python3 -m src.validator --level 2 # Level 2
```

12 對應評分表的自我檢核

評分項	配分	本作業狀態
1 週期任務集 (1-1~1-8)	17	全數通過 (見 §4.1)
2 模型限制 (C1~C23, 含放寬版)	27	全數實作並通過自我檢核
3 放寬假設 (R1~R10)	10	10 條皆由動態排程實際滿足
4 接收測試 (含 4-3 value rate)	11	零星 value rate = 1.0; 附逐 job 理由
5 排程結果	8	JSON 格式相容; 週期 job 全準時
6 評估指標	7	全部指標皆輸出
7 保留策略分析	10	報告分析 + 保留下限 R-reserve
8 動態法 (8-1/8-2/8-3)	10	設計 + 正確性 + 靜/動比較皆具備

自我檢核工具 `python3 -m src.validator --level 2` 會用放寬後的儲能/再生能源限制重驗每一項，並逐條確認動態排程確實滿足 *R1-R10* (例如 *SOC* 是否遵循效率/自放電平衡、再生能源是否守住實際上限、優先序是否成立)，亦即驗證「實作」而非僅「描述」。

第二部分 報告講稿（明日對老師報告）

建議總時長 10–12 分鐘。以下每段對應一張投影片，粗體為口語講稿，項目為提醒重點。

投影片 1：開場與題目定位（約 1 分鐘）

講稿：老師好，我們這組做的是「虛擬電廠的即時排程系統」。核心想法是：把電廠裡的傳統機組、太陽能、電池當成即時系統的『處理器』，把各種有時間限制的用電需求當成『任務』，於是供電調度就變成一個即時排程問題。我們要在 72 小時、每小時一格的時間軸上安排這些處理器供電，同時滿足三個彼此衝突的目標：少漏非週期任務、降低機組成本、提高售電收益。

- 先讓老師建立「處理器 = 發電設備、任務 = 用電需求」的對應。
- 強調三個目標互相衝突，這是後面 Level 2 分析的伏筆。

投影片 2：系統架構與四階段管線（約 1 分鐘）

講稿：整個系統分四個階段：Phase 1 生成週期任務集；Phase 2 用混合整數線性規劃解出日前排程；Phase 3 在排程留下的保留量上做接收測試，處理零星與非週期任務；Phase 4 計算所有效能指標。Level 2 我們在這個基礎上放寬三個假設，並加上一個會隨資訊滾動再最佳化的動態排程器。

- 指著架構圖由左到右講一次資料流。
- 點出 Phase 3 已整合進排程器之後自動執行。

投影片 3：Phase 1 週期任務生成（約 1 分鐘）

講稿：週期任務是用「生成—驗證」迴圈產生的，會反覆重抽直到同時滿足作業所有限制；我們加了固定種子讓它可重現。我們刻意先生成 $d = e$ 的任務和非搶占任務來保證最難滿足的條件。這次產生 8 個週期任務，frame size 是 4，展開後是 38 個 job，超過 30 個；密度 1.31 遠高於 0.7 的下限；frame size 也通過 $2f - \gcd(f, p) \leq d$ 這條。

- 報出關鍵數字：8 個任務、frame= 4、38 個 job、密度 1.31。
- 若問為何是 frame size，說明它保證每個 job 有完整一格可被排進去。

投影片 4：Phase 2 MILP 模型——變數與目標（約 1.5 分鐘）

講稿：日前排程是一個 MILP，我們用 PuLP 加 CBC 一次解出整個 72 小時。決策變數有設備出力 P 、能量分配 k 、機組開關機、電池充放電與 SOC、售電量 Sell，還有需求是否執行的二元變數 x 。目標函數是 α 乘上非週期逾時數，加機組成本，減售電收

益； α 設成 10000，代表每漏一個非週期任務罰一萬元。規格裡的 $\min(1, \cdot)$ 是非線性，我們用二元變數 x 和 u 把它線性化，這樣才能交給 MILP 解。

- 強調「一次解完整個 horizon」是靜態法的特徵，與 Level 2 對照。
- 解釋 $\alpha = 10000$ 把「個數」換算成「金額」才能和成本相加。

投影片 5：Phase 2 模型——23 條限制式（約 1.5 分鐘）

講稿：限制式共 23 條，分成幾群：任務類保證每個 job 在截止前累積執行恰好 e 格、非搶占要連續；機組類包含出力上下限、爬升率、最小開關機時間；再生能源受預測上限約束；儲能類包含充放電上限、SOC 平衡、不可同時充放；最後是每小時的供需平衡 C23——每一格的總發電必須等於用電需求加充電加售電。這條平衡式是整個模型的骨幹。

- 不必逐條念，挑 C23 平衡與 C3 完成性、C5 連續性講透即可。
- 提一句 C16/C18 在 Level 2 會被放寬版取代。

投影片 6：Phase 3 接收測試與保留策略（約 1.5 分鐘）

講稿：排程解完後，市場上沒賣掉的剩餘電力就是我們的「保留量」。接收測試就靠這個保留量來接零星和非週期任務。零星任務是硬截止：只有當它的時間視窗內每一格的保留量都夠它的能量需求、而且能排滿執行格數時，才接受，否則拒絕。非週期任務是軟截止：盡量排，逾時就記成 miss。接受時我們把需求從設備的剩餘出力轉進去、同時把售電量等額扣掉，所以總發電不變、平衡式仍然成立。這次 6 個零星任務全部接受，value rate 是 1.0。

- 講清楚「轉移剩餘、扣售電、平衡不變」這個關鍵手法。
- 報出：零星全接受、value rate 1.0；靜態下 a_2 、 a_5 兩個非週期逾時。

投影片 7：Level 2-A 放寬三個假設（約 1.5 分鐘）

講稿：Level 2 我們放寬三個假設：再生能源預測不一定準、儲能不是理想的、以及工作之間有優先序。最重要的限制是：規格規定可以加參數、改符號，但不能新增決策變數，所以我們所有放寬限制都只用原本的變數來寫。再生能源我們把預測折減 15% 留邊際；儲能我們加了充放電效率 0.95、自放電、循環上限、SOC 相依功率和老化成本；優先序則是強制某個 job 做完另一個才能開始。總共 10 條放寬限制，剛好對到評分的 10 分上限。把參數設回預設，模型就跟 Level 1 完全一樣。

- 反覆強調「沒有新增決策變數」——這是評分硬規定。
- 舉一兩個具體：SOC 平衡加了效率與自放電；老化成本進目標。

投影片 8：Level 2-B 動態排程方法（約 1.5 分鐘）

講稿：動態排程用的是滾動視窗再最佳化。Level 1 是一次解完全程，動態法則是沿時間往前走，在三種觸發點重新最佳化剩下的時程：固定每 24 小時一次、每個零星任務一到就審核、還有實際太陽能跟預測差最大的幾個點。已經執行過的部分會被「釘住」，只重解後面，這樣每次再解都很快，而且機組狀態跟 SOC 都正確接續。已承諾的區塊用「實際」的太陽能出力、看不到的未來用折減預測，所以對已承諾精確、對未來保守。任務能不能接受，看再最佳化能不能在它的視窗內保留足夠剩餘，這就是把接收測試做成 MILP 的可行性問題。

- 強調「釘住前綴 + 只重解尾段」是讓線上再最佳化可解的關鍵。
- 用一句話帶出：硬任務接受了就保證準時完成。

投影片 9：結果——9 輪再最佳化與正確性（約 1 分鐘）

講稿：這次動態排程跑了 9 輪、9 次 MILP，大約 12 秒（單執行緒、可重現）。任務隨時間陸續到達並被接受：例如第 2 小時揭露 s1、a1、a2，第 28 小時揭露 s3、a5，一路到第 66 小時的 s6、a10。最後所有 38 個週期 job、6 個零星 job 都在硬截止前完成，10 個非週期 job 也都在軟截止前完成；硬逾時率 0、軟逾時率 0。每一格的供需平衡和 SOC 範圍我們也都逐格驗證沒有違反。

- 報出：9 輪、9 次解、約 12 秒、全部準時。
- 強調「逐格驗證 0 違反」回應評分 8-2 的正確性要求。

投影片 10：結果——靜態 vs 動態比較（約 1.5 分鐘）

講稿：最關鍵的比較在這裡。靜態法雖然一次看到所有任務，但只承諾一份計畫，結果漏掉 2 個非週期任務，軟逾時率 0.2。動態法隨任務到達再最佳化、並主動保留容量，10 個非週期任務全部準時，軟逾時率變成 0，平均回應時間也從 3.63 降到 2.65。少漏 2 個就省下 2 萬元罰則，這主導了總目標從 -69410 改善到 -83239。代價是：保留容量加上補償太陽能實際短缺，機組成本多了約 1.5 萬；不過較大的發電也讓售電收益增加約 8.9 千。這正好驗證我們前面說的——提高接收率要用機組成本來換，三個目標沒辦法同時最佳化。

- 這是全場重點，務必把「省 2 萬罰則 > 多花 9 千成本」的帳算給老師聽。
- 結論句：動態法在總目標上明顯更好，且權衡關係與理論分析一致。

投影片 11：結語與分工 / AI 協作（約 1 分鐘）

講稿：總結：我們完成了 Level 1 的生成、日前排程、接收測試與評估，也完成 Level 2 的三項假設放寬與動態排程，並用實際數據證明動態法在不漏硬截止的前提下，

把軟逾時降到 0、總目標更好。程式採物件導向、單一職責設計，所有結果都可用固定種子重現。謝謝老師，以下開放提問。

- 若需報告 AI 協作：說明用 AI 輔助建模與程式骨架、自己負責驗證與調參，並交叉檢查每條限制式。
- 預留時間回答提問。

預期問答（備用）

- Q：為何用 MILP 而不是啟發式？ A：MILP 能在 23 條硬限制下保證可行且最佳，且線上再最佳化用 gap 與時間上限即可在秒級內解出。
- Q：動態法的計算成本？ A：本例 9 次解約 12 秒（單執行緒）；釘住前綴讓 presolve 消去已承諾段，再解規模隨時間縮小。
- Q：保證沒有新增決策變數？ A：所有放寬限制（R1–R10）與保留下限都只用 $P, k, SOC, Sell, x$ ，已用自我檢核逐條驗證。
- Q：jitter 動態略增（44.1 $\rightarrow$ 45.8）為何仍較好？ A：jitter 是同任務各 instance 完成時刻的離散度，動態法為保留容量微調了部分週期 job 的完成時刻，但硬截止全數守住、總目標更佳，屬可接受的取捨。

（報告結束）